Aide-mémoire : Classification supervisée

Classification supervisée I

TP nombre d'items de la classe d'intérêt correctement classé.

TN nombre d'items classé dans toutes les autres classes de manière correcte

FP : nombre d'items des autres classes qui sont erronément classés

FN : nombre d'items d'intérêt qui sont classés dans une autre classe.

Acc: **(Tp+tn)/(Tp+Tn+Fp+Fn)** : une métrique qui quantifie globalement le modèle.

- Recal= *(TPR)*, sensitivité, *Power*, *Probability of detection*. **Tp/(tp+fn)** . Le taux de vrais malades trouvés par l'ordi parmi tt les malades
- Spec= *(TNR)*, sélectivité. Le taux de vrais sains trouvés par l'odri parmi tt les sains. **Tn/(Tn+Fp)**
- Prec = **Tp/(fp+tp)**

$$ball\ acc = \frac{specificity + recall}{2}$$

$$Fscore = 2 \cdot \frac{precision \cdot recall}{precision + recall}$$

Classification supervisée II

Validation croisée: partionnement des données du set de test et du set d'apprentissage. (By Aude)

On sépare les données en un certain nombre de groupe (argument cv.k =) et on va utiliser ces groupes à la fois dans le set de test et d'apprentissage.

```
# ADL avec toutes les données
penguins_lda <- ml_lda(data = penguins, species ~ bill_length +
  bill_depth + flipper_length + body_mass)
# Prédiction par validation croisée 10x
set.seed(7567) # Pensez à varier le nombre à chaque fois ici !
penguins_pred <- cvpredict(penguins_lda, cv.k = 10)
# Matrice de confusion
penguins_conf <- confusion(penguins_pred, penguins$species)
plot(penguins_conf)
```

On peut aussi réaliser un summary avec les différentes méthodes (code toujours le même) avec l'argument **sort.by** = pour trier les lignes de la meilleure à la moins bonne valeur (donc pas forcément à chaque fois)

```
summary(penguins_conf, type = c("Recall", "Precision",
"Fscore"), sort.by = "Recall")
```

K plus proches voisins (Par Kelly) **:**

Prédire la classe d'un nouvel individu en regardant les k (impair) individus qui lui ressemble (ou est proche) suivant la distance de Mahalanobis (Euclidienne, Manhattan). Utilise individuellement toutes les données du set d'apprentissage.

```
set.seed(3675)
pima1_knn <- ml_knn(data = pima1, diabetes ~ ., k = 3)
pima1_knn_conf <- confusion(cvpredict(pima1_knn, cv.k = 10), pima1$diabetes)
plot(pima1_knn_conf)
summary(pima1_knn_conf, type = c("Fscore", "Recall", "Precision"))
```

Forêt aléatoire (par Albane) :

créé un ensemble d'arbres décisionnels individuels, dont la variable discriminante à chaque nœud est choisie dans un sous-ensemble aléatoire de toutes les p variables utilisées.

un des algorithmes les plus performants de classification supervisée.

```
set.seed(74356)
pima1_rf <- ml_rforest(data = pima1, diabetes ~ .)
pima1_rf_conf <- confusion(cvpredict(pima1_rf, cv.k = 10), pima1$diabetes)
plot(pima1_rf_conf)
summary(pima1_rf_conf, type = c("Fscore", "Recall", "Precision"))
```

graphique montrant la variation de l'erreur en fonction du nombre d'arbres :

plot(pima1_rf)

graphique présentant l'importance des variables :

randomForest::varImpPlot(pima1_rf)

--> simplifier le modèle (élimination variables moins importantes et diminution nb d'arbres)

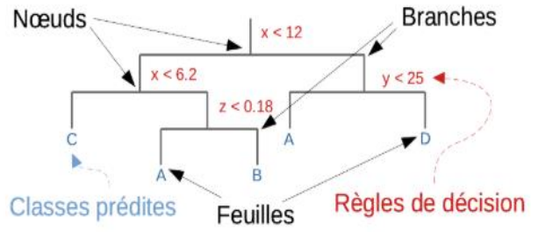
```
set.seed(85436)
pima1_rf2 <- ml_rforest(data = pima1, diabetes ~ glucose + insulin + age + mass + pedigree,
  ntree = 200)
pima1_rf2_conf <- confusion(cvpredict(pima1_rf2, cv.k = 10), pima1$diabetes)
plot(pima1_rf2_conf)
```

Quantification vectorielle (Par Kelly) **:**

Prédire la classe d'un nouvel individu en regardant les k.nn (impair) centroïdes les plus proches. Le set d'apprentissage est résumé en un dictionnaire (codebook) dans lesquels les groupes initiaux sont remplacés par les centroïdes. Définir le taille (size) du codebook.

```
set.seed(846)
pima1_lvq <- ml_lvq(data = pima1, diabetes ~ ., k.nn = 3, size = 30)
pima1_lvq_conf <- confusion(cvpredict(pima1_lvq, cv.k = 10), pima1$diabetes)
plot(pima1_lvq_conf)
summary(pima1_lvq_conf, type = c("Fscore", "Recall", "Precision"))
```

Méthode dicothomique1 : **Le Partitionnement récursif** (by Arthur)



Xpl on choisit X l'age d' un groupe d'enfant. Si X> 12(le seuil), on va vers la gauche si X =< 12 vers la droite.

```
{r·rpart,·record='ROP',·object='zoo2_rpart',·arg='control'}
set.seed(235)
zoo2_rpart·←·rpart(data=zoo2,·Class·~.)
```

```
<!--% Graphique du partitionnement récursif -->
```

```
{r·rpart_plot}
set.seed(63)
plot(zoo2_rpart,·margin·=·0.03,·uniform·=·TRUE)
text(zoo2_rpart,·use.n·=·FALSE)
```

```
{r·rpart_confusion,·record='ROA',·object='zoo2_rpart_conf',·arg='col.freqs'}
set.seed(94785)
zoo2predictbis·←·cvpredict(zoo2_rpart,·cv.k·=·10)
zoo2_rpart_conf·←·confusion·(zoo2predictbis,·zoo2$Class)

plot(zoo2_rpart_conf)
```

Exemple : Préparation, Entrainement et Performance du classifieur (Kelly)

```
set.seed(101121) # Générateur de nombres pseudo-aléatoir
pumpkins_split <- initial_split(pumpkins, prop = 0.8)
# training set
pumpkins_train <- training(pumpkins_split)
# test set
pumpkins_test <- testing(pumpkins_split)
# Afficher pumpkins_split
pumpkins_split
set.seed(3265)
# K plus proches voisins
pumpkins_knn <- ml_knn(data = pumpkins_train, class~., k.nn = 15)
# Partitionnement récursif
pumpkins_part <- ml_rpart(data = pumpkins_train, class~.)
# Forêt aléatoire
pumpkins_rf <- ml_rforest(data = pumpkins_train, class~., ntree = 100)
# K plus proches voisins
predict(pumpkins_knn, pumpkins_test) |> confusion(pumpkins_test$class) |> summary()
# Partitionnement récursif
predict(pumpkins_part, pumpkins_test) |> confusion(pumpkins_test$class) |> summary()
# forêt aléatoire
predict(pumpkins_rf, pumpkins_test) |> confusion(pumpkins_test$class) |> summary()
```

Aide-mémoire : Classification supervisée

Classification supervisée III

Métriques et Proportions: (by Aude)

Remarque: Les formules de métriques vont être utile ici,, la *Précision* et le *Rappel*.

Quand une classe est représentée en sous nombre, cela créé des biais et il faut ré-ajuster les proportions relatives afin que le classifieur ne s'embrouille pas. On va voir comment les modifiés uniquement pour le set d'apprentissage.

Code de rééchantillonnage du set:

```
# Rééchantillonnage du set
set.seed(845)
pima1 %>.%
  group_by(., diabetes) |>
  sample_n(130L, replace = FALSE) ->
  pima1b
table(pima1b$diabetes) |>
  tabularise()
```

```
set.seed(854)
pima1b_rf <- ml_rforest(data = pima1b, diabetes ~ ., ntree = 500)
pima1b_rf_conf <- confusion(cvpredict(pima1b_rf, cv.k = 10), pima1b$diabetes)
summary(pima1b_rf_conf)
```

On peut aussi manipuler les poids des individus en les multipliant ou en les divisants. **UNIQUEMENT POUR LE SET D'APPRENTISSAGE.** On sépare d'abord les données en set d'apprentissage et de test.

```
# Séparer d'abord set d'apprentissage et set de test
set.seed(9856)
pima1_split <- initial_split(pima1, 2/3, strata = diabetes)
# Récupération du set d'apprentissage
pima1_train <- training(pima1_split)
# Récupération du set de test
pima1_test <- testing(pima1_split)
table(pima1_train$diabetes) |> # Tableau mal balancé
  tabularise()

 # Pondération double pour les cas positifs en les dupliquant
 resample_idx <- over_undersample(pima1_train$diabetes,
   weights = c(neg = 1, pos = 2))
 pima1_train <- pima1_train[resample_idx, ]
 table(pima1_train$diabetes) |> # Tableau mieux balancé sans perte
   tabularise()
```

Probabilité a priori: Échantillonnage aléatoire de taille raisonnable pour estimer quelles sont les proportions relatives des classes dans les données à classer en déploiement. Sont déterminées grâce à une table de contingence du jeu de données initial.

Correspondent toujours à la proportion observée de chaque niveau de la variable facteur d'intérêt dans le set d'apprentissage.

```
pima_prior_tab <- table(pima$diabetes) / nrow(pima) # Table de contingence
pima_prior <- structure(as.numeric(pima_prior_tab), names = names(pima_prior_tab))
pima_prior # Vecteur numerique nommé
```

On peut avoir des probabilités a priori venant directement du web

Dans {mlearning}, les probabilités *a priori* peuvent être injectées dans l'objet **confusion** pour corriger nos métriques en faveur de valeurs plus réalistes. Reprenons un cas concret pour l'illustrer : notre set d'apprentissage `pima1b` aux proportions modifiées et dont les métriques ont été calculées par validation croisée en absence de set de test. Nous pouvons les corriger, et en même temps, les rendre plus comparables avec les métriques *non biaisées* calculées sur `pima1` en procédant comme suit :

```
prior(pima1b_rf_conf) <- pima_prior
summary(pima1b_rf_conf)
```

À retenir

Si vous êtes amené à modifier les proportions des différentes classes dans votre set d'apprentissage (pratique conseillée si les proportions sont trop différentes d'une classe à l'autre), n'oubliez pas de repondérer si nécessaire via `prior()`, afin d'avoir des métriques plus représentatives des performances de votre classifieur en déploiement. **Ne comparez jamais entre elles des métriques calculées avec des probabilités *a priori* différentes !**

Courbe ROC (BY Arthur)

La **courbe ROC (pour "Receiver Operating Characteristic")** est une courbe qui représente le comportement **de notre classifieur à deux classes pour tous les seuils de détection possibles. En prenant des seuils de 0 à 1 (ou 100%), nous balayons toutes les possibilités.**

À chaque seuil, nous pouvons calculer le taux de vrais positifs et le taux de faux positifs. La courbe ROC représente ces résultats avec le taux de faux positifs sur l'axe *X* et le taux de vrais positifs sur l'axe *Y*. Pour illustrer cela, nous allons calculer la courbe ROC pour notre classifieur forêt aléatoire pima1_rf.

Tout d'abord, il nous faut **obtenir les (pseudo-)probabilités** *a posteriori***.** Selon les algorithmes ce sont réellement des probabilités, ou alors, des nombres entre 0 et 1 qui ont des propriétés similaires. Dans le cas de la forêt aléatoire, c'est la proportion des arbres qui ont voté pour la classe.

```
## Solution ##
library(ROCR)
# 1) Formater les prédictions pour ROCR
pred_obj <- prediction(breast_pred[,"malignant"], breast$class == "malignant"
# 2) Calculer les performances avec les 2 métriques tpr et fpr
perf <- performance(pred_obj, "tpr", "fpr")
# 3) Tracer le graphique ROC
plot(perf); abline(a = 0, b = 1, lty = 2)
```

Déterminer aire sous la courbe

```
Code R   Start Over   Hints
1 pROC::auc(breast$class,  breast_pred [, "benign"])
2
```

Le trait pointillé représente ce que donnerait un classifieur de référence qui classe en `pos` ou `neg` au hasard, et donc, il obtient autant de faux positifs que de vrais positifs quel que soit le seuil choisi. En dessous, nous ferions moins bien que le hasard. Au-dessus, notre classifieur est meilleur. Et plus nous nous rapprochons du coin supérieur gauche du graphique avec notre courbe, meilleur sera notre classifieur (plus haut taux de vrais positifs pour plus bas taux de faux positifs). Ainsi le choix du seuil peu se faire pour un point de la courbe qui s'approche le plus du coin supérieur gauche. Ici, nous pourrions conserver le seuil qui génère un peu moins de 20% de faux positifs dans notre set de test.

Une façon de quantifier globalement les performances de notre classifieur, quel que soit le seuil, est de **calculer l'aire sous cette courbe**. Pour un classifieur au hasard de référence, elle sera de 0.5 puisque l triangle inférieur délimité par le trait pointillé représente la moitié de l'aire totale du graphique. Plus l'aire sous la courbe (**AUC** ou *Area Under the Curve* en anglais) se rapprochera de un, meilleur sera notre classifieur. Voici une des façons de calculer l'AUC dans R :

Réseau de neurones artificiels (by Albane)

Modèle constitué de :

- **Plusieurs couches interconnectées i :**

1ère couche = attributs mesurés sur les individus

Couches intermédiaires = paramétrage du système d'apprentissage

Dernière couche = renseigne les classes à identifier

- **Noeuds j = neurones artificiels**

Connexion entre neurones caractérisée par des **seuils Wij** (de 0 à 1) --> permettent de déterminer la classe à laquelle un individu appartient

Transfert du "signal" : modélisé par des fonctions logistiques variant entre ces 2 extrêmes (0 et1).

Un réseau de neurones simple à une seule couche cachée est disponible avec la fonction `ml_nnet()` dans {mlearning}. L'utilisation est similaire aux autres fonctions du package :

```
set.seed(85460)
pima1_nnet <- ml_nnet(data = pima1, diabetes ~ .)
```

```
pima1_nnet_conf <- confusion(cvpredict(pima1_nnet, cv.k = 10), pima1$diabetes)
plot(pima1_nnet_conf)
summary(pima1_nnet_conf)
```

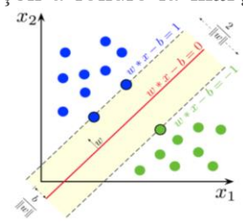
Apprentissage profond (*Deep Learning*)

- **Pilier de l'intelligence artificielle** : forme moderne et avancée des réseaux de neurones.
- **Structure complexe** : réseaux composés de centaines, voire de milliers de neurones artificiels interconnectés en plusieurs couches.
- **Puissance de calcul** : exploite les capacités des ordinateurs actuels pour traiter de grandes quantités de données.
- **Grande capacité d'apprentissage** : permet aux réseaux de s'adapter à des situations complexes.
- **Coût informatique élevé** : nécessite une grande puissance de calcul. Des processeurs **GPU (cartes graphiques puissantes)** sont généralement nécessaires pour entraîner les modèles à une vitesse raisonnable.
- **Exemple d'application : MNIST** :
  - Jeu de données historique contenant des dizaines de milliers de chiffres manuscrits numérisés.
  - Objectif : créer un classifieur capable de reconnaître les chiffres de 0 à 9.
  - Le Deep Learning permet aujourd'hui d'atteindre des performances égales, voire supérieures, à celles d'un humain pour reconnaître des chiffres manuscrits.

À retenir : Le Deep Learning repose sur de grands réseaux de neurones capables d'apprendre à résoudre des problèmes complexes. Il nécessite toutefois une puissance de calcul importante et permet d'obtenir des performances remarquables dans des domaines comme la reconnaissance d'écriture manuscrite ou la conduite autonome.

Machine à vecteurs supports (Kelly)

- Les SVM : algorithmes qui déterminent, pour un problème à **deux classes**, le **meilleur hyperplan** (droite) qui sépare les classes dans l'espace des variables (x1 et x2). Contrairement à l'ADL, qui utilise toutes les données pour placer la frontière, le SVM n'utilise que les individus proches de la frontière : les **vecteurs supports**. La frontière est placée de façon à rendre la **marge** la plus large possible.



- L'approche par **noyau (kernel)** permet au SVM de tracer des frontières non linéaires en projetant les données dans un espace où elles sont linéairement séparables. La frontière devient une droite (hyperplan) dans l'espace projeté, mais reste courbe dans l'espace d'origine. Plusieurs noyaux : **linéaire** (frontière linéaire), **polynomial** (frontière courbe type polynôme), **radial** (frontière flexible), **sigmoïde** (frontière en S).

- Le SVM n'est défini que pour **deux classes**. Pour traiter un problème **multiclasse**, on le ramène à plusieurs problèmes à deux classes : choisir une classe cible et la comparer à toutes les autres regroupées **(macro)** ou comparer individuellement deux à deux **(micro)**.

```
set.seed(960412)
# Machine à vecteurs supports
derma_svmr <- ml_svm(data = derma, class ~ .)
# Machine à vecteurs supports avec un noyau linéaire
derma_svml <- ml_svm(data = derma, class ~ ., kernel = "linear")
# Réseaux de neurones artificiels
derma_nnet <- ml_nnet(data = derma, class ~ ., size = 12)
set.seed(367)
# Machine à vecteurs supports
cvpredict(derma_svmr, cv.k = 10) |> confusion(derma$class) |> summary()
# Machine à vecteurs supports avec un noyau linéaire
cvpredict(derma_svml, cv.k = 10) |> confusion(derma$class) |> summary()
# Réseaux de neurones artificiels
cvpredict(derma_nnet, cv.k = 10) |> confusion(derma$class) |> summary()
```

- Avec noyau polynomial

```
set.seed(150)
pima1_svm3 <- ml_svm(data = pima1, diabetes ~ ., kernel = "polynomial",
  degree = 2, coef0 = 1)
pima1_svm3_conf <- confusion(cvpredict(pima1_svm3, cv.k = 10), pima1$diabetes)
plot(pima1_svm3_conf)
summary(pima1_svm3_conf)
```